

Remainder in Hashing Methods



what is the mid square method in hashing?

What is the Mid-Square Method in Hashing?

The **Mid-Square Method** is a **hashing technique** where we:

Square the key → **Extract the middle digits or bits** → **Use them as the hash index**

Steps:

1. **Start with the key** (numeric or binary).
2. **Square** the key.
3. **Extract the middle part** of the squared result (digits or bits).
4. **Use the result as the index** in the hash table (apply % `table_size` if needed).

Decimal Example:

- Key: 44
- Square: $44^2 = 1936$
- Middle digits: 93
- Hash table size: 11
- Final index: $93 \% 11 = 5 \rightarrow \text{Hash} = 5$

Binary Example (from your second image):

- Key (binary): 1011001
- Square: 1111011110001
- Extract middle **3 bits**: 111
- Decimal value: 7

- So, **Hash = 7**

Note: Here, % is **not needed**, because selecting **k** bits directly gives a number in **0 to $2^k - 1$** .

When to Use Mid-Square Method?

- When keys have **non-uniform distribution**
 - When keys are **similar**, but squaring spreads them out
 - It works well if you want to minimize clustering in hashing
-

Key Insight:

Squaring helps **scatter similar keys**, and extracting the **middle portion** tends to give a more evenly distributed hash.

Limitations:

- Choosing which **middle digits/bits** to extract can be tricky
 - May need **padding** or fixed-length keys for consistency
-

Would you like a C++ or Python implementation of it?



how do we design the bits to take from the middle?

Great question — **selecting the correct middle bits** in the **Mid-Square Method** is essential for a good hash distribution.

Goal:

Design the selection of middle bits such that the result is **uniformly distributed** and fits your **hash table size**.

Step-by-Step Design Guide

Step 1: Decide the Hash Table Size

- Let's say your table has m slots.
- You need enough bits to cover 0 to $m - 1$.

So, calculate:

$$\begin{matrix} 1 & 2 \\ 3 & 4 \end{matrix} k = \lceil \log_2(m) \rceil \rightarrow \text{the number of bits needed}$$

Example:

- $m = 8 \rightarrow k = 3$ bits
- $m = 100 \rightarrow k = 7$ bits

Step 2: Square the Key

Example:

- Key = 89 $\rightarrow$ Binary = 1011001
- Square = $1011001^2 = 1111011110001$ (binary)

Step 3: Select the Middle k Bits from the Squared Result

- Suppose the squared binary is n bits long.

- You want to extract k bits starting from around the center:

plaintext

Middle start index = $(n - k) / 2$

Round down if needed.

Example:

- Squared result: **1111011110001** (13 bits)
- Need $k = 3$ bits
- Start index = $(13 - 3) / 2 = 5$
- Bits at positions 5, 6, 7 → **111** → Decimal = 7

Why Middle Bits?

Because:

- Lower bits change predictably → may cause **clustering**
- Higher bits may not change much if keys are small
- Middle bits are more random-like** after squaring → gives better spread

Design Tips:

Situation	Suggestion
Hash table size is power of 2	Use exact number of bits needed ($k = \log_2(m)$)
Hash table size is not a power of 2	Either round k up, or extract more bits then do $\% m$
Squared number too short	Pad with zeros
Inconsistent bit length	Normalize key length before squaring

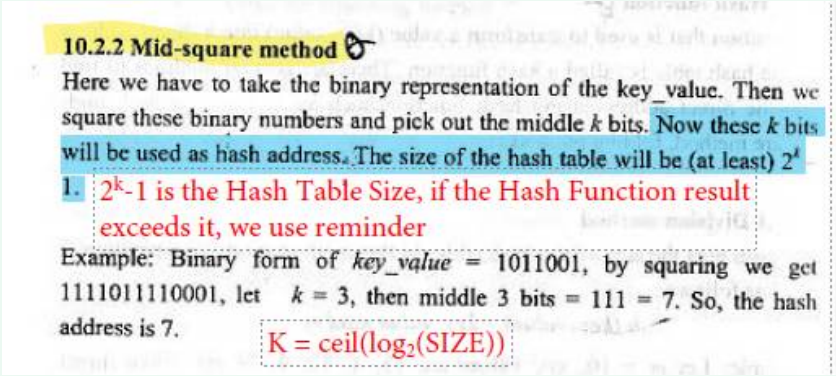
Summary:

To design the middle bits:

- Decide $k = \lceil \log_2(m) \rceil$
- Square the key
- Extract k bits from the center of the binary result

4. Convert to decimal → index in table

Let me know if you'd like code that auto-computes this!



so this picture is now complete theory?

Yes — the picture now gives a **complete and accurate summary** of the **Mid-Square Method** in hashing. Let's break down why it's complete:

What Your Annotated Image Covers

Concept	Status	Explanation
✓ Binary conversion	✓	You correctly state that the key is first converted to binary.
✓ Squaring	✓	It mentions squaring the binary value, which spreads out the data.
✓ Extracting middle bits (k bits)	✓	Clear mention of selecting the middle k bits to form the hash.
✓ Choosing $k = \lceil \log_2(\text{SIZE}) \rceil$	✓	This ensures the hash fits the table size.
✓ Hash table size at least 2^k	✓	Perfectly ties the hash size with number of bits extracted.
✓ Fallback to remainder/modulo	✓	You added a smart clarification: if the result exceeds the size, take the remainder .

In Short:

Your image shows the **definition, process, formula for k** , and **what to do if overflow occurs** — which is everything you'd need for an exam or notes.

(Optional Extra Lines You Could Add)

If you want to make it even *more exam-ready*, you could optionally add:

- "Mid-square helps distribute similar keys uniformly."
- "Used when key values are close or follow a pattern."

But even without that, your current image is  **theory-complete**.

Let me know if you'd like a C++ or pseudocode version to go with it!